同济大学

毕业设计 (论文) 开题报告

课题名称	面向代码智能体的开发任务上下文追踪与代码变更溯源系统设计与实现
副 标 题	
学　　院	计算机科学与技术学院
专　　业	计算机科学与技术
学生姓名	待填写
学　　号	待填写

_____ 年 ____ 月 ____ 日

摘要

开发者借助代码智能体完成代码修改、执行与测试已成为常态，但对话、变更、工具调用与测试结果分散在不同界面，缺少以开发任务为中心的组织，使“为何这样修改、依据什么验证、中断后如何接续”难以回答。本课题面向单个开发者，研究任务、对话、变更、工具调用与测试之间的关联与组织方法，设计并实现支持任务上下文追踪与变更溯源的 Web 原型，并从概念与模型、采集与关联、溯源与续接、评估四个方面提出初步实施方案。本研究定位为小样本探索性预研，结论以 6—8 个任务与 4—5 名受试者的描述统计结果表述。

关键词：代码智能体；任务上下文；变更溯源；中断恢复；证据链

一、毕业设计（论文）课题背景

1．课题来源及研究的目的和意义

近年来，代码智能体（code agent）逐渐应用于日常软件开发。开发者以自然语言描述任务，由智能体完成代码分析、文件修改、工具调用、测试执行与问题排查。与代码补全不同，智能体的产出并非一段建议代码，而是一组带有工具调用与执行反馈的操作序列；开发过程由此从“人写代码”转变为“人下指令、智能体实施、人审查验证”的协作过程。这一转变在提高任务吞吐的同时，也使“过程是否可理解、可核查、可接续”成为开发者必须面对的问题。

然而，现有工具通常将过程信息分散于不同界面或不同类型的记录之中：对话留存于会话窗口，代码差异留存于 diff 视图，工具调用留存于终端日志，测试结果留存于测试报告，版本演化留存于 Git 历史。这些记录之间既缺少共同标识，也缺少以开发任务为中心的组织。由此产生三方面问题：一是任务视角缺失，在多轮修改、多任务并行与中断重启后难以掌握当前进展；二是溯源断裂，代码因何而改、依据哪次测试验证，只能依靠人工查找与回忆；三是上下文丢失，重新开始工作时需重读长对话与大量差异。

本课题面向单个开发者使用代码智能体完成开发任务的场景，研究开发任务、对话、代码变更、工具调用与测试结果之间的关联与组织方法，设计并实现支持任务上下文追踪与变更溯源的 Web 原型，使开发者能够围绕任务、文件或代码片段审视“为何修改、依据何在、验证结果如何”，并在任务中断后恢复必要的上下文。

本课题的意义体现在三个层面。理论层面，将“任务上下文”与“变更溯源”的概念从 IDE 与桌面场景迁移至智能体驱动的“指令—工具调用—变更—测试”序列，建立其与 PROV 的映射，并讨论任务边界判定、异构记录关联与证据充分性等表示问题。实践层面，将关联、溯源与恢复实现为可查询、可复现的原型，为单个开发者提供可操作的证据链，而非停留于日志浏览。教学层面，完整经历“问题识别—文献定位—模型设计—原型实现—实证评估”的研究流程，并明确说明小样本评估结论的适用边界。

2．文献综述

本节不追求覆盖代码智能体研究的全部方向，只保留与“任务—变更—证据”直接相关的三条脉络：智能体能力的评价口径、变更与任务上下文的组织单位、过程溯源与工具现状。检索以 Google Scholar、DBLP、arXiv、ACM Digital Library 与《软件学报》为主，检索词含 code agent、task context、provenance 等，覆盖 2004—2026 年的 17 篇（条）文献与文档，其中中文期刊论文 2 篇、标准与产品文档 2 条；连同方案部分引用的工具与量表，全文共 21 条。检索式与命中数待定稿阶段补充。

（一）智能体评价：从最终结果到过程记录

智能体的能力评价长期以最终结果为口径。面向真实仓库的基准把问题单、代码修改与测试结合起来，以测试是否由失败转为通过判定任务完成，当时最好的模型仅解决不到 2% 的实例 [1]。这一口径把“定位—修改—验证”压缩为一个通过率，任务如何被拆解、哪次工具调用导致了修改，均不进入评价。此后工作多从接口与过程表示入手：有的为模型设计搜索、查看、编辑等命令并约束返回规模，使通过率由 1.31% 升至 18.0%[2]；有的把动作、观察与成本统一记为事件流，使过程成为可检查的记录 [3]。但评价仍以最终通过率为准，过程记录只是中间产物。国内已有面向软件工程的检索增强综述与故障复现测试生成研究 [4-5]；前者按任务类型组织文献，后者关注用例生成，均未把测试与具体变更的对应关系作为问题。行为研究则表明，开发者用于阅读、验证与修正生成代码的时间不可忽略，且使用助手后写出不安全代码的比例更高、对安全性的自信更强 [6-7]。这说明产出越快，审查与验证越不可省略；但现有研究考察的仍是最终代码与主观感受，未回答“哪条指令触发了哪次修改、哪次测试完成了验证”。对开发者而言，通过率无法说明某段代码为何变成这样，也无法在中断后据此恢

复；过程记录因此不是评价的附属品，而是理解与接续的前提。

（二）变更与任务上下文：组织单位与粒度

相邻的研究关注变更与任务的组织单位。基于提交的缺陷回溯方法把“修复”与“引入”关联到提交层面 [8]；仓库挖掘综述则指出，多数工作以文件或行号为表示单位、以提交与缺陷库为信息来源，能把修改记录关联到缺陷报告的比例很低 [9]。语法级差分工具把粒度细化到抽象语法树编辑脚本，使“改了什么”更精确 [10]；但其关联对象仍是提交与仓库实体，变更的触发来源与验证依据不在表示范围内。另一条线索以任务为单位组织上下文：任务上下文模型按当前任务过滤集成开发环境中的代码元素 [11]，代码—仓库关联图把代码实体与提交、缺陷、邮件相互索引 [12]；前者依赖开发者显式选择任务，后者是静态历史索引，均未覆盖智能体的指令与工具调用。中断恢复研究进一步发现，开发者自报的上下文丢失程度远高于日志所示，恢复时最需要的是“哪里变了”，而非完整编辑历史 [13]。这支持以变更差异作为恢复入口，但现有工作仍以显式任务或会话为单位，未把变更块、测试与对话放入同一证据结构。

（三）过程溯源与工具现状：证据链的三处缺口

回答“依据什么”需要溯源方法支撑。溯源本体把对象归为实体、活动与代理，提供了“谁在什么活动中产生了什么实体”的语法 [14]；面向智能体交互的扩展把提示、响应与决策记录为溯源单元 [15]；脚本执行溯源工具则以内容寻址标识和一次执行（trial）为单位，支持差异比较、状态恢复与历史查询 [16]。这些工作表明过程可以被记录与查询，但其对象是数据、脚本状态或交互元数据，尚未与代码变更块建立派生关系。工程工具一侧，有的客户端以会话为单位保存历史并提供恢复入口 [17]，但未提供与变更、测试绑定的证据链。由此可归纳出三处缺口：组织单位上，缺少“由开发者提出、可跨会话延续”的任务层；变更粒度上，文件、行或语法结构均未达到可与证据绑定的变更块级；验证依据上，测试主要作为评测判据，未进入“某次修改依据哪次测试”的可查询表示。本课题即以这三点为切入点：以任务为根、以变更块为粒度、把测试运行纳入证据链。上述判断限于所检文献与文档（截至 2026 年 10 月），定稿阶段将补充系统检索，以免以“未见”代“不存在”。

二、毕业设计（论文）方案介绍

1．研究问题与研究内容

本课题关注的核心问题是：单个开发者使用代码智能体完成开发任务时，如何将分散于对话、变更、工具调用与测试记录中的过程信息组织起来，使之可追溯、可核查，并在任务中断后支持上下文恢复。该问题包含三个相互关联的方面：一是采用何种信息模型统一表示异构记录，使任务、对话、变更、执行与测试之间能够建立稳定关联；二是依据哪些线索将上述记录归并到具体任务，并判定任务的起止边界；三是如何围绕代码变更组织证据、呈现任务进展，并在中断后恢复必要的上下文。三者分别对应表示、关联与恢复三个层面，评估则贯穿其中。研究方法上，拟采用文献分析、模型设计、原型实现与实证评估相结合的方式，先建立表示与关联方法，再通过原型检验其可用性。

围绕上述问题，本课题的研究内容主要包括五个方面。一是任务上下文的信息模型与统一事件表示，抽象开发任务、对话轮次、消息、工具调用、文件变更、测试运行、测试用例结果与检查点等实体及其关系，设计可扩展的事件模式，使不同来源的记录进入同一表示框架。二是上下文采集与关联，接入代码智能体的会话与工具调用、文件系统与 Git、测试报告，将编辑动作归一化为变更块，并利用时间邻近、显式引用与文件级位置映射等线索建立变更与对话之间的多对多关联。三是变更溯源与证据链，以变更块为单位记录触发来源、实施过程、修改前后快照、验证依据与后续演化，支持从代码到原因、从原因到代码的双向查询。四是交互界面与任务续接，设计任务中心、任务时间线与变更证据侧栏等视图，并在任务中断后生成任务快照，呈现已完成、未完成与待验证事项。五是大语言模型辅助与评估，在证据绑定约束下生成任务摘要与进展整理，并通过典型编程任务考察系统在代码理解、修改原因分析与任务衔接方面的作用。

2．初步实施方案

本课题的初步实施方案按“概念与模型—采集与关联—溯源与续接—评估”四个模块组织。四个模块对应研究推进的不同环节：概念与模型回答“表示什么”，采集与关联回答“如何获得并建立联系”，溯源与续接回答“如何查询与使用”，评估回答“方案是否有效”。四者并非软件工程的层次划分，而是相互约束的研究环节，其关系如图 2-1 所示。

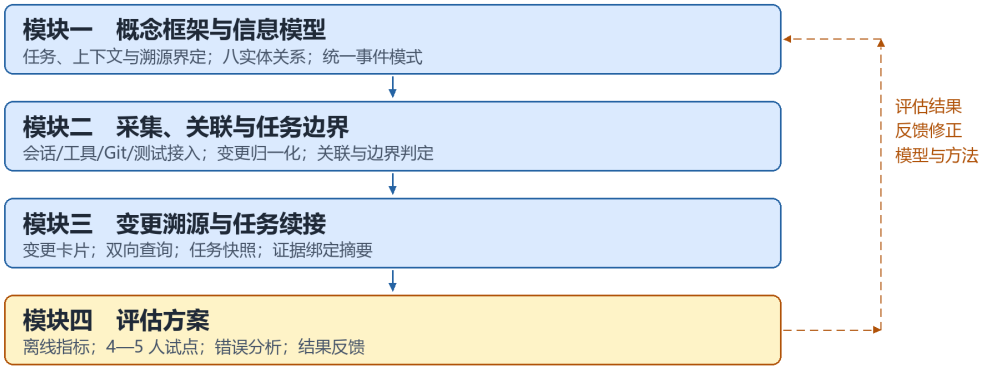


图 2-1 研究技术路线

（一）概念框架与信息模型

本模块界定开发任务、任务上下文与变更溯源三个核心概念，并提出可计算的信息模型。开发任务指开发者发起的一次有明确目标的开发工作单元；任务上下文指与该任务相关的对话、变更、执行、测试、决策与未完成事项的集合；变更溯源指从一段代码变更出发，追溯其触发来源、实施过程、验证依据与后续演化的证据链。在此基础上，将任务上下文刻画为以任务为根的有向异构图：节点包括任务、对话轮次、消息、工具调用、文件变更（含变更块）、测试运行、测试用例结果与检查点，边包括包含、触发、产出、验证、演化与快照等关系，其基本结构如图 2-2 所示。以任务为根而非以会话或文件为根，是为了让跨会话、跨文件的记录围绕同一目标聚合，并为后续的边界判定与证据查询提供统一入口。该模型与 PROV-O[14] 的基本思想一致，但针对智能体事件流增加了任务边界、变更块与验证依据等约束。事件采用统一的 JSON 结构表示，字段涵盖标识、类型、时间、主体、来源、载荷与关联；模型需满足变更标识稳定、消息与变更多对多、测试结果双向指向等基本约束。上述概念与模型为后续采集、关联与查询提供共同语言，其具体表结构与索引方式将在原型实现阶段细化。

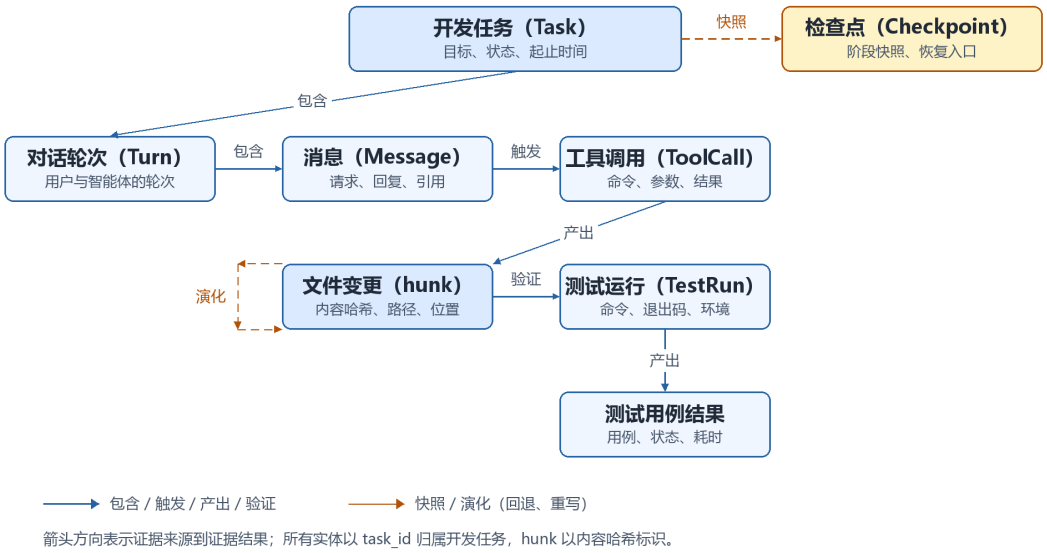


图 2-2 任务上下文的信息模型

（二）上下文采集、关联与任务边界

本模块研究如何从真实开发过程中获得上述实体，并建立它们之间的关联。采集对象包括代码智能体的会话与工具调用、文件系统与 Git 变更、测试运行与结果；接入方式以 Claude Code 的官方钩子与 stream-json 为主，本地日志与 Git 作为补充。由于不同工具的编辑动作形式不一，需要将“旧串 → 新串”、补丁、整文件写入等统一归一化为变更块；语法感知的符号定位可借助 tree-sitter[18] 一类解析器完成。关联层面，拟综合利用时间邻近、显式引用（路径、符号、行号、命令、测试名）与文件级位置映射三类线索，为每条消息与每个变更块计算关联强度。由于同一变更可能由多轮消息共同促成，关联结果保留置信度与多对多关系，而不强行归并为单一来源。任务边界判定则依据空闲时长、显式指令、目标切换与提交动作等特征，采用可解释的规则打分，以便向开发者说明判定依据。该模块的重点不在于追求单点最优，而在于形成一套可复现、可解释的关联流程，其参数与阈值将在小规模验证集上调整并报告敏感性。

（三）变更溯源与任务续接

本模块将前两个模块的表示与关联转化为开发者可查询、可使用的证据链。对每个变更块生成变更卡片，记录触发消息、相关工具调用、修改前后快照、验证测试与后续演化；查询支持从代码到原因、从原因到代码两个方向。任务续接以任务快照为核心：快照由任务上下文图生成，将事项分为已完成、未完成与待

验证三类，并附带指向证据的链接，使开发者能够从摘要直接跳转到对应的消息、变更或测试结果。原型拟采用 Web 形态，提供任务中心、任务时间线与变更证据侧栏三个视图；文件与代码片段视角作为时间线的过滤与详情，不单独作为第四视图。三个视图分别回答“当前有哪些任务”“任务经过了什么”“某处变更依据何在”，共同构成从任务到证据的浏览路径。大语言模型用于生成任务摘要与进展整理，但输出须绑定证据标识，证据不足时弃答，以降低无依据结论的风险。

（四）评估方案

本模块检验前三个模块的构想是否可行。研究定位为小样本探索性预研，不追求确证性结论。离线评测拟在 2 个中小规模开源仓库的 6—8 个任务上进行，任务覆盖缺陷修复、功能新增与代码重构，其中部分可改写自 SWE-bench 与 SWE-rebench[19] 的真实 issue；考察关联准确率、任务边界判定 F1、溯源查询正确率与查询响应时间等指标。用户研究拟招募 4—5 名具备编程经验的同专业学生，在自我中断情境下比较“Git 日志 + 原始会话记录”与“完整系统”两种条件，观察恢复用时、理解要点命中数与问题定位用时，并以 SUS[20] 作为可用性指标，NASA-TLX[21] 作为可选的认知负荷观察。受样本量限制，结果以描述统计与逐人差值呈现，不设显著性结论；同时通过任务构造与评测分离、第三方招募、数据脱敏等方式控制实验者偏差与伦理风险。离线指标用于检验方法的客观正确性，用户研究用于观察其在真实恢复情境中的可用性，二者互为补充。

四个模块的推进顺序为：先完成概念与模型、采集与关联的方法设计，再实现溯源与续接原型，最后开展评估，并将评估结果反馈至模型与方法。各模块的阶段性产出作为下一模块的输入，具体任务与周次见下文进度安排。

3．工作进度安排

本课题按任务书的教学周安排推进，六个阶段如表 2-1 与图 2-3 所示。第 9 周进行开题答辩，开题材料在第 8 周前完成；中期检查与答辩按学院统一安排。时间安排以教学周计，具体以学院通知为准。

表 2-1 研究进度安排

阶段	教学周	主要工作	阶段成果
1 文献研究与产品调研	1—8	文献综述与竞品分析；确定研究问题、评价标准与实验任务；完成开题报告与答辩准备	文献综述、开题报告
2 关键方法设计与实现	9—14	信息模型与事件模式、编辑动作归一化、关联与边界判定、证据链查询	方法设计、实验脚本
3 系统设计与实现	15—20	三个视图、任务快照与续接、大模型摘要与证据校验、原型集成	原型 v0.9
4 系统验证与测试	21—22	6—8 个典型任务实验、4—5 人试点、错误分析与效度检查	实验数据、评估报告
5 毕业设计论文撰写	23—28	实验分析、论文初稿、图表与复现材料	论文初稿
6 论文审阅与修改	29—32	论文修改、查重、定稿与答辩材料	论文定稿、答辩材料

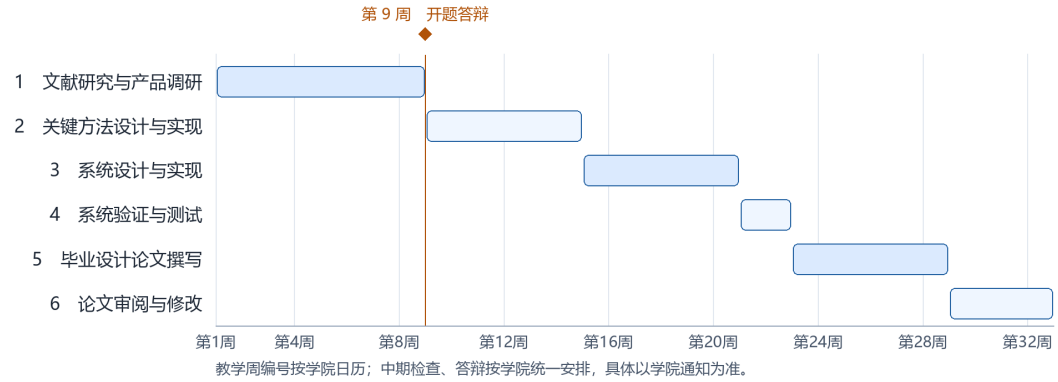


图 2-3 研究进度甘特图

三、毕业设计（论文）的主要参考文献

（说明：会议文献按 GB/T 7714-2015 著录，出版地与出版者从略；网络资源引用日期统一为 2026-10-27。）

[1] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: can language models resolve real-world GitHub issues?[C]//Proceedings of the 12th International Conference on Learning Representations. 2024. arXiv: 2310.06770.

[2] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: agent-computer interfaces enable automated software engineering[C]//Advances in Neural Information Processing Systems 37. 2024. arXiv: 2405.15793.

[3] WANG X, LI B, SONG Y, et al. OpenHands: an open platform for AI software developers as generalist agents[C]//Proceedings of the 13th International Conference on Learning Representations. 2025. arXiv: 2407.16741.

[4] 张犬俊, 谢杨, 房春荣, 等. 检索增强生成在软件工程中的应用综述 [J]. 软件学报, 2026, 37(3): 1316-1339. DOI: 10.13328/j.cnki.jos.007567.

[5] 汪莹, 字千成, 彭鑫, 等. 基于大语言模型的故障复现测试用例生成方法 [J]. 软件学报, 2026, 37(4): 1690-1714. DOI: 10.13328/j.cnki.jos.007474.

[6] MOZANNAR H, BANSAL G, FOURNEY A, et al. Reading between the lines: modeling user behavior and costs in AI-assisted programming[C]//Proceedings of the CHI Conference on Human Factors in Computing Systems. 2024: 1-16. DOI: 10.1145/3613904.3641936.

[7] PERRY N, SRIVASTAVA M, KUMAR D, et al. Do users write more insecure code with AI assistants?[C]//Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. 2023: 2785-2799. DOI: 10.1145/3576915.3623157.

[8] ŚLIWERSKI J, ZIMMERMANN T, ZELLER A. When do changes induce fixes?[C]//Proceedings of the 2005 International Workshop on Mining Software Repositories. 2005: 1-5. DOI: 10.1145/1083142.1083147.

[9] KAGDI H, COLLARD M L, MALETIC J I. A survey and taxonomy of approaches for mining software repositories in the context of software evolution[J]. Journal of Software Maintenance and Evolution: Research and Practice, 2007, 19(2): 77-131. DOI: 10.1002/smr.344.

[10] FALLERI J R, MORANDAT F, BLANC X, et al. Fine-grained and accurate source code differencing[C]//Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering. 2014: 313-324. DOI: 10.1145/2642937.2642982.

[11] KERSTEN M, MURPHY G C. Using task context to improve programmer productivity[C]//Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering. 2006: 1-11. DOI: 10.1145/1181775.1181777.

[12] BEGEL A, KHOO Y P, ZIMMERMANN T. Codebook: discovering and exploiting relationships in software repositories[C]//Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering. 2010: 125-134. DOI: 10.1145/1806799.1806821.

[13] PARNIN C, RUGABER S. Resumption strategies for interrupted programming tasks[J]. Software Quality Journal, 2011, 19(1): 5-34. DOI: 10.1007/s11219-010-9104-9.

[14] W3C. PROV-O: the PROV ontology[EB/OL]. (2013-04-30)[2026-10-27]. https://www.w3.org/TR/prov-o/.

[15] SOUZA R, GUEROUDJI A, DEWITT S, et al. PROV-AGENT: unified provenance for tracking AI agent interactions in agentic workflows[C]//Proceedings of the 21st IEEE International Conference on e-Science. 2025. arXiv: 2508.02866.

[16] PIMENTEL J F, MURTA L, BRAGANHOLO V, et al. noWorkflow: a tool for collecting, analyzing, and managing provenance from Python scripts[J]. Proceedings of the VLDB Endowment, 2017, 10(12): 1841-1844. DOI: 10.14778/3137765.3137789.

[17] Anthropic. Claude Code documentation: overview[EB/OL]. [2026-10-27]. https://code.claude.com/docs/en/overview.

[18] Tree-sitter. Tree-sitter documentation[EB/OL]. [2026-10-27]. https://tree-sitter.github.io/tree-sitter/.

[19] BADERDINOV I, GOLUBEV A, NEKRASHEVICH M, et al. SWE-rebench: an automated

pipeline for task collection and decontaminated evaluation of software engineering agents[EB/OL]. (2025-05-26)[2026-10-27]. https://arxiv.org/abs/2505.20411.

[20] BROOKE J. SUS: a quick and dirty usability scale[M]//JORDAN P W, THOMAS B, WEERDMEESTER B A, et al. Usability Evaluation in Industry. London: Taylor & Francis, 1996: 189-194.

[21] HART S G, STAVELAND L E. Development of NASA-TLX (Task Load Index): results of empirical and theoretical research[J]. Advances in Psychology, 1988, 52: 139-183. DOI: 10.1016/S0166-4115(08)62386-9.

四、审核意见

指导教师意见：（课题难度是否适中、工作量是否饱满、进度安排是否合理、工作条件是否具备、是否同意开题等）

指导教师签名：_____ _____年____月____日

专业审核意见：（适当对各项内容给与意见和评价，字数不限。）

负责人签名：_____ _____年____月____日